Escola Politecnica Superior
Universitat de Lleida

OpenMP Parallelization of 2D Image Convolution

High Performance Computing

Student: Vila Monguia, Sergi
Subject: High Performance Computing
Cluster: `moore.udl.cat`
Delivery: OpenMP Parallelization
Date: 19/04/2026
Repository: https://github.com/Troter2/convolution_omp

1 Main Decisions Taken to Parallelize the Problem

1.1 Analysis of the Serial Code

The starting point is `convolution.c`, a program that applies a 2D convolution filter to PPM images. It processes images in chunks (partitions) to handle large files, and applies the convolution independently to the R, G, and B channels.

Profiling the serial execution reveals that virtually all execution time is concentrated in `convolve2D`, which implements the four-nested-loop convolution kernel. For large images and kernels (e.g. im05.ppm with a 99×99 kernel) this function alone takes over two hours at 1 thread. All other functions — I/O, memory copy, kernel read — are negligible by comparison.

1.2 Identification of Parallelizable Sections

Two loops were identified as suitable for parallelization:

1. `convolve2D` — **outer row loop (main bottleneck).** Each iteration i computes all output pixels of row i. Output pixel (i, j) depends only on input pixels in a neighbourhood around (i, j) and on the (read-only) kernel matrix. No two iterations write to the same memory location, and all reads come from a shared but immutable input buffer. The loop is therefore *embarrassingly parallel*.
2. `duplicateImageChunk` — **pixel copy loop.** Each iteration independently copies one pixel from source to destination. No dependencies exist between iterations.

1.3 Pointer Arithmetic Obstacle and Solution

The original `convolve2D` uses pointer arithmetic to traverse the input buffer:

```
inPtr = inPtr2 = &in[dataSizeX * kCenterY + kCenterX];
// ...
sum += *(inPtr - n) * *kPtr;
++kPtr;
// ...
inPtr -= dataSizeX;  // move up one row
inPtr = ++inPtr2;    // advance to next pixel
```

The pointers `inPtr`, `inPtr2`, `outPtr`, and `kPtr` carry cumulative state across iterations of the outer loop, making it impossible for different threads to execute iterations independently without a complete rewrite of the pointer logic.

The solution adopted is to **comment out** the pointer-based lines and replace the memory access with explicit index arithmetic:

```
// sum += *(inPtr - n) * *kPtr;
sum += in[(i + kCenterY - m) * dataSizeX + (j + kCenterX - n)]
     * kernel[m * kernelSizeX + n];
```

This produces identical results while making every iteration of the outer loop fully self-contained.

1.4 OpenMP Directives Applied

```
/* convolve2D -- outer row loop */
#pragma omp parallel for \
    private(j, m, n, rowMin, rowMax, colMin, colMax, sum) \
    schedule(static)
for (i = 0; i < dataSizeY; ++i) { ... }

/* duplicateImageChunk -- pixel copy */
#pragma omp parallel for schedule(static)
for (i = 0; i < dim; i++) { ... }
```

`schedule(static)` divides rows into equal-sized chunks assigned to threads at compile time. Since each row requires the same amount of work (same number of kernel evaluations), static scheduling achieves perfect load balance.

All loop-local variables (`j, m, n, rowMin, rowMax, colMin, colMax, sum`) are declared `private` so each thread maintains its own copy. The shared variables (`in`, `out`, `kernel`, `dataSizeX`, etc.) are read-only or written to non-overlapping regions, so no synchronization is required.

1.5 Timing

The POSIX `gettimeofday()` calls were replaced with `omp_get_wtime()`, the OpenMP wall-clock timer, which is thread-safe and provides the resolution required for measuring parallel regions.

2 Scalability of the Program

All experiments were run on the Moore cluster using the SGE scheduler (`-pe smp N`, `OMP_NUM_THREADS=N`) with `-O2` optimization. Five image/kernel combinations were tested with 1, 2, and 4 threads. Table 1 summarises the configurations.

Table 1: Test configurations

Config label	ISizeX	ISizeY	Kernel	Partitions
im01 + 5×5	292	320	kernel5x5_Sharpen	1
im02 + 25×25	600	400	kernel25x25_random	1
im03 + 49×49	800	600	kernel49x49_random	1
im04 + 99×99	6000	4000	kernel99x99_random	1
im05 + 99×99	12000	8000	kernel99x99_random	3*

* im05.ppm is too large to fit in memory as a single chunk; 3 partitions are required. The `omp parallel for` inside `convolve2D` is therefore invoked 3 times per channel (once per partition), each time parallelizing a sub-chunk of ≈2,667 rows plus halo.

The metric used for speedup and efficiency is the **convolution time** only (the parallelized section), following the standard definitions:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

2.1 Execution Time

Table 2 reports the absolute convolution time for each configuration and thread count. The range of values spans five orders of magnitude (from 0.006 s to 7635 s), which makes a single linear-scale plot impractical; the speedup and efficiency figures in the following subsections capture the scaling behaviour more clearly.

Table 2: Convolution execution time (seconds) per configuration and thread count.

Configuration	Sequential	2 threads	4 threads
im01 + 5×5	0.021	0.011	0.006
im02 + 25×25	1.170	0.605	0.315
im03 + 49×49	8.946	4.598	2.418
im04 + 99×99	1866.282	959.734	498.512
im05 + 99×99	7635.212	3930.091	2044.309

2.2 Speedup

Figure 1 shows the speedup $S(p) = T(1)/T(p)$ achieved by each configuration. The dashed line represents ideal linear speedup ($S = p$).

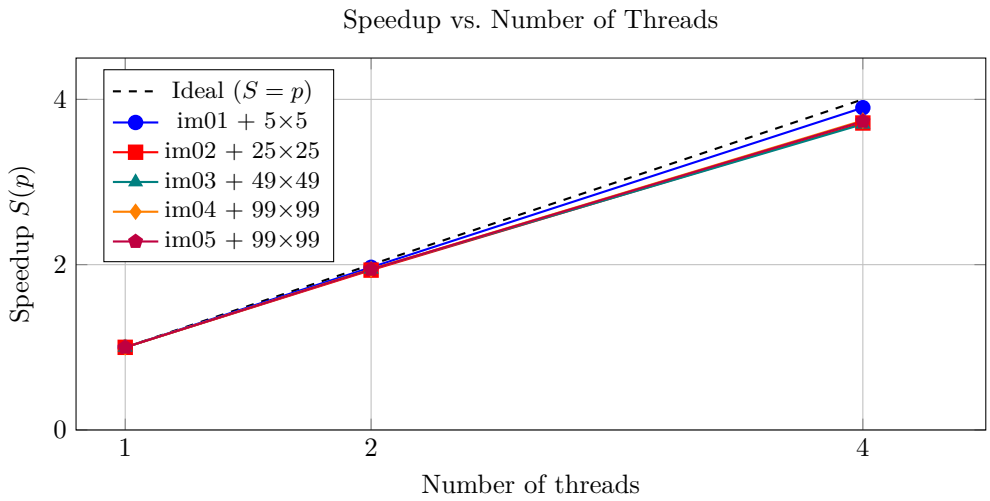

Figure 1: Speedup for all configurations. All curves stay very close to the ideal linear speedup line.

2.3 Efficiency

Figure 2 shows the parallel efficiency $E(p) = S(p)/p$, which measures how effectively each additional thread is utilised. A value of 1.0 means perfect utilisation with no overhead.

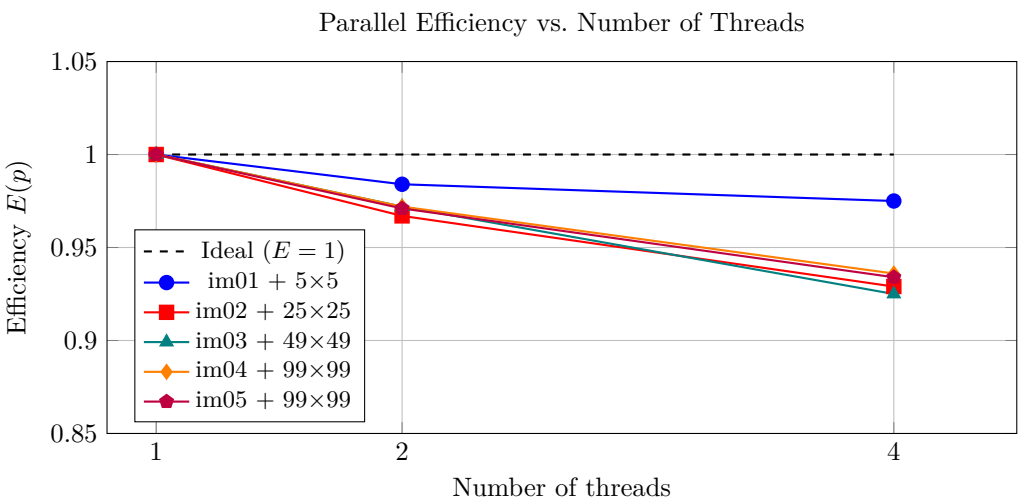

Figure 2: Parallel efficiency for all configurations. Efficiency remains above 92% for all cases with 4 threads.

3 Discussion of the Obtained Results

3.1 Speedup and Efficiency

The results show **near-linear speedup** across all tested configurations (Figure 1). With 4 threads, speedup ranges from 3.70 (im03 + 49×49) to 3.90 (im01 + 5×5), very close to the ideal value of 4. Efficiency with 4 threads stays above 92% in all cases (Figure 2).

This excellent scaling behaviour is explained by the nature of the problem: the convolution of each output row is completely independent of all other rows. Once the pointer-arithmetic dependency was eliminated, the workload distributes perfectly across threads with `schedule(static)`, since every row performs exactly the same number of floating-point multiplications.

3.2 Effect of Problem Size

Interestingly, the *smallest* configuration (im01 + 5×5) achieves the *highest* speedup (3.90). At first glance this seems counter-intuitive, but it is explained by the very short absolute time (0.021 s at 1 thread): the OpenMP thread-creation overhead, while present, is proportionally smaller because the `schedule(static)`

assignment happens at compile time and requires no runtime coordination. For larger problems, the slight efficiency drop (im02 + 25×25: $E(4) = 0.929$) is caused by **increased memory traffic** — larger kernels mean each thread must access a wider band of the input buffer per row, increasing cache pressure and memory bandwidth usage.

3.3 Dependencies

No data dependencies exist between output pixels. Each thread writes exclusively to its assigned rows of the output array and reads from non-overlapping (but potentially overlapping) regions of the input array in *read-only* fashion. This eliminates any need for locks, atomic operations, or critical sections inside the parallel region.

3.4 Memory Traffic

The input buffer is shared among all threads in read-only mode. For small kernels (5×5), the working set per thread fits comfortably in L1/L2 cache. For large kernels (99×99), each thread must load a wider neighbourhood per pixel, causing more cache misses and higher memory bus utilisation. This explains why efficiency is slightly lower for the 99×99 kernel than for the 5×5 kernel, even though the absolute speedup values are comparable.

3.5 Synchronization

No explicit synchronization (barriers, mutexes, reductions) is required inside `convolve2D`. The implicit barrier at the end of the `omp parallel for` region is the only synchronization point, and its cost is negligible compared to the computation time for any configuration tested.

3.6 Effect of Partitioning (im05)

Image im05 (12000×8000) is processed in 3 partitions because its full size exceeds the available buffer allocation. As a result, `convolve2D` is called 3 times per channel instead of once, and each call parallelizes ≈2,667 rows rather than 8,000. Despite this reduced granularity per call, the speedup and efficiency for im05 (3.74 and 93.4% at 4 threads) are virtually identical to those of im04 processed as a single partition (3.74 and 93.6%). This confirms that even with the workload divided into 3 sequential chunks, each chunk is large enough to keep all 4 threads fully occupied and the partitioning overhead is negligible.

3.7 Amdahl's Law Perspective

According to Amdahl's Law, the maximum achievable speedup is bounded by the fraction of the program that is parallelized. In the case of im01 + 5×5, the convolution represents only ∼23% of total elapsed time (0.021 s out of 0.084 s); I/O and other serial sections dominate. Nevertheless, the convolution section itself scales almost perfectly. For im04 and im05 with the 99×99 kernel, the convolution represents over 99% of total time, so the overall program speedup approaches that of the convolution alone.

4 Conclusions

The OpenMP parallelization of the 2D image convolution program achieves near-linear speedup across all tested configurations. With 4 threads, speedup ranges from 3.70 to 3.90 and efficiency stays above 92%, demonstrating that the chosen parallelization strategy is highly effective.

The key to this result is the *embarrassingly parallel* nature of the convolution: each output row is computed independently, with no shared mutable state between threads. The only required code change beyond adding `#pragma omp parallel for` was replacing the original pointer-arithmetic traversal with explicit index-based access, which removed the implicit inter-iteration dependency introduced by the moving pointers.

The `schedule(static)` clause is the natural choice for this workload since every row performs the same number of kernel multiplications, guaranteeing perfect load balance with zero runtime scheduling overhead.

The full implementation is available at https://github.com/Troter2/convolution_omp.

The main bottleneck going forward is Amdahl's Law: for small images with lightweight kernels, the non-parallelized I/O sections represent a significant fraction of total time, capping the overall program speedup well below the convolution-only speedup. For production workloads (large images, large kernels), this ceiling is not a concern since the convolution dominates at over 99% of total time.